UNIVERSIDAD DE MONTEMORELOS

Facultad de Ingeniería y Tecnología
Universidad de Montemorelos

Universidad de Montemorelos

Título:

Informe de proyecto integrador:

ODOMETRÍA DIFERENCIAL

Materia:

Estándares Web

Alumno

Jesús Alexis Arias Ricárdez

Matrícula

1240195

30/04/2026

¿Qué es y de qué trata el proyecto?

La odometría es básicamente la forma en que un robot sabe dónde está dentro de un espacio, sin usar GPS ni cámaras. Lo que hace es llevar la cuenta de cuánto ha girado cada rueda y, a partir de eso, calcular su posición y orientación actuales.

Este proyecto simula ese proceso completo: un servidor genera los datos de los encoders de las dos ruedas (como si fueran sensores reales), y una interfaz web los consume, hace los cálculos y dibuja la trayectoria del robot en tiempo real.

El proyecto se estructuró de la siguiente manera:

```
Odometría-Diferencial/
├── server.js       → Servidor Node.js que simula los encoders
├── index.html      → Dashboard visual con mapa, gráficas y log
└── package.json    → Dependencias del proyecto
```

¿Cómo funciona paso a paso?

1. **Simulación de encoders**

Lo primero que nos preguntamos fue: ¿Cómo generamos datos de encoders sin tener un robot físico?

La solución fue crear un servidor HTTP sencillo en Node.js que simula el comportamiento de dos encoders de rueda (izquierda y derecha). Cada **50 milisegundos** el servidor ejecuta un ciclo donde decide, de forma pseudoaleatoria, si cada rueda avanza un tick o no.

La lógica de decisión funciona la siguiente:

- Se generan dos números aleatorios entre 1 y 100 (uno por rueda).
- Si el número es menor a 50 → el tick se cuenta **si el número es impar**.
- Si el número es mayor o igual a 50 → el tick se cuenta **si el número es par**.

Esto produce un avance irregular y realista, parecido a lo que haría un robot en movimiento real con pequeñas variaciones.

Para convertir los ticks en distancia, se usa la siguiente fórmula:

```
incremento (m/tick) = (π × diámetro de la rueda) / ticks por vuelta
```

Con los parámetros del proyecto:

- Diámetro de rueda: 14 cm (0.14 m)
- Ticks por vuelta: 20
- Incremento resultante: ≈ 0.021991 m/tick

El servidor expone estos datos vía una API en http://localhost:3000 que devuelve un JSON con los ticks acumulados y la distancia recorrida por cada rueda.

2. Cálculo de posición

La interfaz web consulta la API del servidor cada 50 ms y, con cada respuesta, calcula la nueva posición del robot usando las ecuaciones clásicas de odometría diferencial.

Cada ciclo hace lo siguiente:

a) Calcula cuánto avanzó cada rueda desde el ciclo anterior (dL y dR).

b) Calcula la **distancia lineal** recorrida por el robot (promedio de ambas ruedas):

```
dc = (dL + dR) / 2
```

c) Calcula el **cambio de orientación** (cuánto giró el robot) usando el ancho entre ruedas (B):

```
dθ = (dR - dL) / B
```

d) Actualiza la **posición** en el plano XY usando el ángulo promedio del movimiento:

```
posX += dc × cos(θ - dθ/2)
posY += dc × sin(θ - dθ/2)
```

La razón de usar θ - dθ/2 (el ángulo a la mitad del movimiento) es para que la trayectoria sea más suave y precisa, en lugar de calcularla solo con el ángulo inicial o el final del ciclo.

El ancho entre ruedas (B) es un parámetro configurable desde la interfaz, con un valor por defecto de **20 cm**.

3. Visualización

El dashboard está construido solo con HTML, CSS y JavaScript. Usa **Chart.js** para las gráficas y Canvas API para el mapa de trayectoria.

Lo que se puede ver en pantalla:

- **Mapa 2D:** dibuja la trayectoria recorrida por el robot y su orientación actual. Tiene zoom y modo "siguiendo al robot" o vista libre.

- **Gauges de rueda:** muestran los ticks acumulados y las revoluciones completadas de cada rueda (izquierda en naranja, derecha en rojo).
- **Gráficas en tiempo real:**
  - Distancia acumulada por rueda.
  - Orientación (θ) a lo largo del tiempo.
  - Diferencia de ticks entre ruedas (indica si el robot está girando).
- **Panel de estadísticas:** posición X/Y, ángulo actual, distancia total recorrida, velocidad por rueda.
- **Log de eventos:** registro en tiempo real de cada tick detectado, advertencias de posible derrape (cuando la diferencia entre ruedas es mayor a 5 ticks), y eventos del sistema.

Tecnologías usadas

- **Node.js** — servidor de simulación de encoders
- **HTML / CSS / JavaScript** — interfaz completa sin frameworks de frontend.
- **Chart.js** — gráficas en tiempo real.
- **Canvas API** — renderizado del mapa de trayectoria.